

Manejo de Excepciones

1. Conceptos Básicos de Excepciones

2. Manejo de Excepciones en Hardware

3. Manejo de Excepciones en Software

4. Excepciones y Sistemas Operativos

5. Prácticas de Programación

6. Seguridad

1. Conceptos Básicos de Excepciones

- Definición: **Una excepción es un evento anómalo que ocurre durante la ejecución de un programa y que interrumpe el flujo normal de las instrucciones.**
- Tipos de Excepciones:
 - - Excepciones de Hardware
 - - Excepciones de Software
 - Excepciones del Sistema Operativo

2. Manejo de Excepciones en Hardware

- Proceso:
 1. **Detección:** El hardware detecta una condición de excepción, como una operación aritmética inválida.
 2. **Interrupción:** El procesador interrumpe la ejecución normal y salta a una rutina de manejo de excepciones predefinida.
 3. **Manejo:** La rutina de manejo de excepciones resuelve el problema o notifica al sistema operativo para su manejo.
 4. **Reinicio:** Se intenta reanudar la ejecución del programa o se termina si no es posible continuar.

Ejemplo: División por Cero

3. Manejo de Excepciones en Software

Ejemplo en Lenguajes de Programación
(JAVA):

- Try-Catch Blocks (Bloques Try-Catch)

```
try {  
    int resultado = 10 / 0; // Esto lanzará una  
    ArithmeticException  
} catch (ArithmeticException e) {  
    System.out.println("Error: División por  
cero.");  
} finally {  
    System.out.println("Este bloque se ejecuta  
siempre.");  
}
```

Para entender que hace este código en JAVA

1. Estructura de control try-catch-finally:

1. El bloque try contiene el código que se intentará ejecutar.
2. Si ocurre una excepción (un error durante la ejecución), el bloque catch captura esa excepción y permite manejar el error de forma controlada.
3. El bloque finally siempre se ejecuta, sin importar si se lanzó una excepción o no.

2. Detalles del código:

1. En el bloque try, intentas realizar una división: $10 / 0$. Dado que en matemáticas la división por cero no está permitida, esto generará una **excepción aritmética** (ArithmeticException).
2. El bloque catch captura la excepción ArithmeticException y ejecuta el código que está dentro, mostrando el mensaje: Error: División por cero..
3. El bloque finally se ejecuta siempre, independientemente de si hubo un error o no, y muestra: Este bloque se ejecuta siempre..

3. Salida esperada:

1. El mensaje Error: División por cero. será impreso porque la excepción es capturada.
2. El mensaje Este bloque se ejecuta siempre. también se imprimirá debido al bloque finally.

¿Y en en Phytton?

try:

 resultado = 10 / 0 # Esto lanzará una excepción ZeroDivisionError

except ZeroDivisionError:

 print("Error: División por cero.")

finally:

 print("Este bloque se ejecuta siempre.")

4. Excepciones y Sistemas Operativos

- Función del SO:
 - Interrupciones y Señales
 - Protección de Memoria
- Ejemplo: Page Fault
 - Un Page Fault (o fallo de página) es una excepción que ocurre en un sistema operativo cuando un proceso intenta acceder a una página de memoria que no está presente en la memoria física (RAM) en ese momento. En un sistema operativo que implementa la gestión de memoria virtual, la memoria del sistema se organiza en páginas, las cuales pueden estar almacenadas en la memoria física o en el disco (generalmente en un archivo de paginación o swap).

5. Prácticas de Programación

- Buenas Prácticas:
 - Validación de Datos
 - Manejo de Excepciones
 - Recursos
- Ejemplo en Python:

```
try {  
    with open('archivo.txt', 'r') as file:  
        data = file.read()  
} except FileNotFoundError:  
    print('Archivo no encontrado.')  
finally:  
    print('Operación de archivo finalizada.')
```


¿Explicamos ese código?

En resumen, este código hace lo siguiente:

- **Intenta abrir un archivo:** Busca el archivo "archivo.txt" y lo abre para leer su contenido.
- **Maneja errores:** Si el archivo no existe, muestra un mensaje de error.
- **Lee el archivo:** Si el archivo se encuentra, lee todo su contenido y lo guarda en la variable data.
- **Siempre finaliza:** Independientemente de si se encontró el archivo o no, imprime un mensaje indicando que la operación ha finalizado.

¿Para qué sirve este código?

- Este tipo de código es muy común en la programación y se utiliza para:
- **Leer archivos:** Para procesar la información contenida en archivos de texto, CSV, etc.
- **Validar la existencia de archivos:** Antes de realizar operaciones sobre un archivo, es importante asegurarse de que exista.
- **Manejar excepciones:** Permite que el programa continúe ejecutándose incluso si ocurre un error, evitando que se detenga abruptamente.

6. Seguridad

- Importancia:
 - Evitar Explotaciones
 - Manejo Adecuado
- Ejemplo en JAVA de validación de entradas:

```
try {  
    int numero =  
    Integer.parseInt(userInput);  
} catch (NumberFormatException e) {  
    System.out.println('Entrada inválida:  
no es un número.');
```

¿Que hace este código en JAVA?

- **En resumen:**

Este código intenta convertir una cadena de texto a un número entero. Si la conversión es exitosa, el valor numérico se almacena. Si la cadena no representa un número, se captura la excepción y se muestra un mensaje de error.

- **Ejemplo práctico:**

Imagina que este código forma parte de un programa que pide al usuario que ingrese su edad. Si el usuario ingresa "25", la conversión se realiza correctamente y el programa puede continuar con otras operaciones. Sin embargo, si el usuario ingresa "veinte" o cualquier otro texto que no sea un número, se mostrará el mensaje de error y el programa no se bloqueará.

- **¿Para qué sirve este tipo de código?**

Este tipo de código es fundamental para validar la entrada del usuario y evitar que el programa se cuelgue si se ingresan datos incorrectos. Al manejar las excepciones de forma adecuada, se hace que el programa sea más robusto y resistente a errores.

FIN

